

SeniorHelper Application – Milestone 3 Documentation

CEN4025C Software Development II | Spring 2026

1. Meeting Information

Date & Time: 02/04/26 6:30 – 8:30 PM (**120 Minutes**)
Location: In Person & WebEx

Date & Time: 02/11/26 6:00 – 8:30 PM (**150 Minutes**)
Location: WebEx

2. Meeting Overview

(02/04)

- Completed individual module 3 project, determined team to-do's.
- Team needs to update the Spring26 repository to reflect progress made in the individual project.

(02/11)

- Began work on Senior Helper application to reflect progress made in the individual project.
 - Login & Registration components completed, connected /education to available modules.
 - Interaction flows completed for Login/Logout, Education, Appointment.
 - Each component team will need to fill-in their interaction flows by Friday 2/13 by 11:00 PM.
-

3. App Contributor Roles & Teams

Name	Role	Functional & Component Sub-Team
Lauren Knotts	Team Lead, Full-stack Dev., Scribe	Education System
Baudelaire Messago Ngumouo	Full-stack Dev.	User Mgmt. (Carelink)
Dayne Allen-Wright	Full-stack Dev.	Appointment
Dewey Mccrary	Full-stack Dev.	Appointment, User Auth.
Tamia Jenkins	Full-stack Dev.	User Mgmt. (Carelink)
Ricardo Serrano	Full-stack Dev., Scribe	Education System, User Auth.

4. Team To-Do's

All team members are responsible for the following tasks.

- Submit design artifacts showing how Angular will communicate with backend APIs – sketches, diagrams, wireframes, or text visuals.
 - Determine API endpoints needed based on milestone 2 UI mockup.
 - Add onto UI from milestone 2 with notes on which APIs each screen uses. (ex. /api/modules).
 - Notes about JWT tokens for user/system authentication (possibly with a step-by-step rundown).
 - Start integrating into the Spring26 repository, catching up to the individual module projects.
-

5. Decisions

- Team deadlines for submissions making it into the milestone document is **2/16/26 11:00 PM**
- Course Project Milestone document will be submitted **no later than 2/17/26 at 12:00 PM**

6. Update Spring 2026 SeniorHelper Repository to Reflect Individual Project Progress - Code

Component	Responsible Members	Completion Status
User Authentication	Dewey, Ricky	In Progress
User Management (Carelink / Connections)	Baudelaire, Tamia	No Update
Appointment	Dayne, Dewey	In Progress
Education System	Lauren, Ricky	In Progress
Navigation Bar & Footer	Lauren	Complete
Main Dashboard	Group	In Progress

7. Data Interaction Flow Examples (Text Visual)

Login / Logout Interaction Flow (Dewey & Ricky)

1. User lands on the login page.
The Angular router redirects to **/login**, which renders LoginComponent.
Files: [app.routes.ts](#)
2. User submits the login form.
The form calls onSubmit(loginForm) in the component.
Files: [login.component.html](#)
3. LoginComponent.onSubmit validates input and calls the API.
If the form is valid, calls AuthService.login({ username, password }).
Files: [login.component.ts](#)
4. AuthService.login POSTs to the backend.
Endpoint: POST <http://localhost:8080/api/auth/login>
Body: { "username": "...", "password": "..." }
Files: [auth.service.ts](#)
5. AuthController.login handles the POST.
It receives a LoginRequest and calls AuthService.login(req).
Files: [AuthController.java](#)
6. AuthService.login validates credentials against the database.
Loads user via UserRepository.findByUsername, re-hashes the incoming password with stored salt, and compares hashes.
Files: [AuthService.java](#), [UserService.java](#)
7. If valid, a token is issued and returned.
Token is a UUID, stored in an in-memory map with a 12-hour TTL.

Response body: { "token": "...", "message": "..." }

Files: AuthService.java

8. Frontend stores the token and navigates to /home.
saveToken uses localStorage if “Remember me” is checked, otherwise sessionStorage.
Files: Login.component.ts, auth.service.ts
9. All future API calls automatically include Authorization: Bearer <token>.
Interceptor adds the header, TokenAuthFilter validates it on the backend.
Files: auth.interceptor.ts, TokenAuthFilter.java

Appointment Interaction Flow (**Dayne & Dewey**)

1. Access Calendar

1. User clicks **Calendar** on the dashboard, Angular navigates to /calendar.
2. Appointment component loads appointmentService
3. Frontend sends HTTP GET request to /api/appointments.

2. Add Appointment

1. User clicks a date on Calendar
2. Angular highlights date and brings appointments for that date on right side of screen
3. User enters info into Add appointment section
4. Frontend send HTTP Update request to /api/appointments

3. Delete Appointment

1. User clicks a date on Calendar
2. Angular highlights date and brings appointments for that date on right side of screen
3. User clicks delete button on available appointment
4. Frontend send HTTP Delete request to /api/appointments

Connections (Carelink) Interaction Flow (**Baudelaire & Tamia**)

1. View Carelink Page

1. User clicks **Connections** on the dashboard, Angular Router navigates to /connections.
2. CarelinkComponent loads.
3. Frontend determines logged-in user role (Caregiver or Senior).
4. Frontend sends HTTP GET request to one of the following endpoints:
 - If **Caregiver** → GET /api/care-links/by-caregiver?caregiverId={id}
 - If **Senior** → GET /api/care-links/by-senior?seniorId={id}

2. Display Carelink Data

1. Backend receives request in CareLinkController.
2. Spring Security evaluates @PreAuthorize authorization rules.
3. Controller calls appropriate service method based on user type (Senior or Caregiver):
 - CareLinkService.forCaregiver(caregiverId) OR
 - CareLinkService.forSenior(seniorId)
4. Service retrieves CareLink entities from CareLinkRepository.
5. Frontend displays returned Carelink relationships, including associated user information and connection details.

3. Create Carelink Connection

1. User initiates creation of a caregiver–senior link.
2. Frontend sends HTTP POST request, CareLinkController receives request.
3. Spring Security evaluates:
 - @permissionChecker.hasPermission(principal, seniorId)
4. Controller calls CareLinkService.link(caregiverId, seniorId).
5. Service performs business logic:
 - 5.1 Validates caregiver exists
 - 5.2 Validates senior exists
 - 5.3 Creates new CareLink entity
 - 5.4 Saves entity using CareLinkRepository
6. Frontend refreshes Carelink list using appropriate GET endpoint.

4. Delete Carelink Connection

1. User initiates removal of a caregiver–senior link.
2. Frontend sends HTTP DELETE request:
 - DELETE /api/care-links?caregiverId={id}&seniorId={id}
3. CareLinkController receives request.
4. Spring Security evaluates:
 - @permissionChecker.hasPermission(principal, seniorId)
5. Controller calls CareLinkService.remove(caregiverId, seniorId).
6. Service removes CareLink entity via CareLinkRepository.
7. Frontend refreshes Carelink list using appropriate GET endpoint.

Education Interaction Flow (Lauren & Ricky)

1. View All Modules

1. User clicks **Education** on the dashboard, Angular Router navigates to **/education**.
2. **EducationComponent** loads and calls **ModuleService.getAllModules()**.
3. Frontend sends **HTTP GET** request to **/api/modules**.

2. Display Module List

1. Backend returns module list.
2. UI displays each module with its associated number, title, description, and a 'Start Learning' or 'Continue Learning' button (mapped to whether a user has completed 1 or more lesson within that module).

3. Open a Specific Module

1. User selects a module, Angular Router navigates to **/education/{moduleId}**.
2. **ModuleComponent** loads and calls **ModuleService.getModule(moduleId)** -> GET **/api/modules/{moduleId}**

4. Display Module Content

1. Backend returns module details, lessons, quiz data, and progress.
2. UI displays module page with lesson list, completion status (check mark/hollow circle), progress bar(?), and quiz access.

5A. Load & Complete Lesson

3. User selects a lesson from individual module screen, Angular Router navigates to **/education/{moduleId}/lessons/{lessonId}**.
 - Frontend requests lesson content: GET /api/modules/{id}/lessons/{lessonId}
4. Backend returns lesson data and return/complete buttons.
 - Content is loaded from both lesson-content.json and data.sql.
5. User selects complete indicator button “Mark Complete,” backend updates user completion and recalculates progress. Returns user to **/education/{moduleId}**.

5B. Take Quiz

1. User selects **Take Practice Quiz** from individual module screen, router navigates to **/education/{moduleId}/quiz**.
 - Frontend requests module quiz data (GET /api/modules/{id}/quiz).
2. Backend returns quiz, user submits quiz, backend scores and returns results, UI updates applicable progress modifiers.
3. User remains on quiz results page and must navigate back via Education icon in navbar.

8. UI Communication with Backend APIs

Created in Figma, attached in separate doc titled ‘CEN4025C M3 – API Communication’